Serverless AI Chatbot with AWS Lex & Lambda

By Oyeleke Ismail

AWS Solutions Architect & .NET Developer

Using Amazon Lex, AWS Lambda, Python, and Kommunicate.io

---

1. Project Overview

This project demonstrates the deployment of a Serverless AI Assistant designed to enhance user engagement on my personal portfolio website. Unlike standard contact forms, this conversational interface provides instant, 24/7 responses to recruiters and hiring managers.

The solution leverages **Amazon Lex** for Natural Language Understanding (NLU), **AWS Lambda** (Python) for backend logic and fulfillment, and **Kommunicate.io** for a seamless custom UI integration on the frontend.

---

2. Problem Statement

Static portfolio websites often suffer from passive user interaction:

- **Delayed Communication:** Recruiters must email and wait for a reply to get basic information like resumes or availability.
- **Engagement:** Text-heavy "About Me" pages are often skimmed or ignored.
- **Navigation Friction:** Finding specific details (e.g., "Do you know AWS?") requires searching through multiple pages.

Solution: This project addresses these challenges by embedding an intelligent agent that answers questions instantly, guiding visitors to the information they need most (Skills, Projects, Contact Info) in a conversational format.

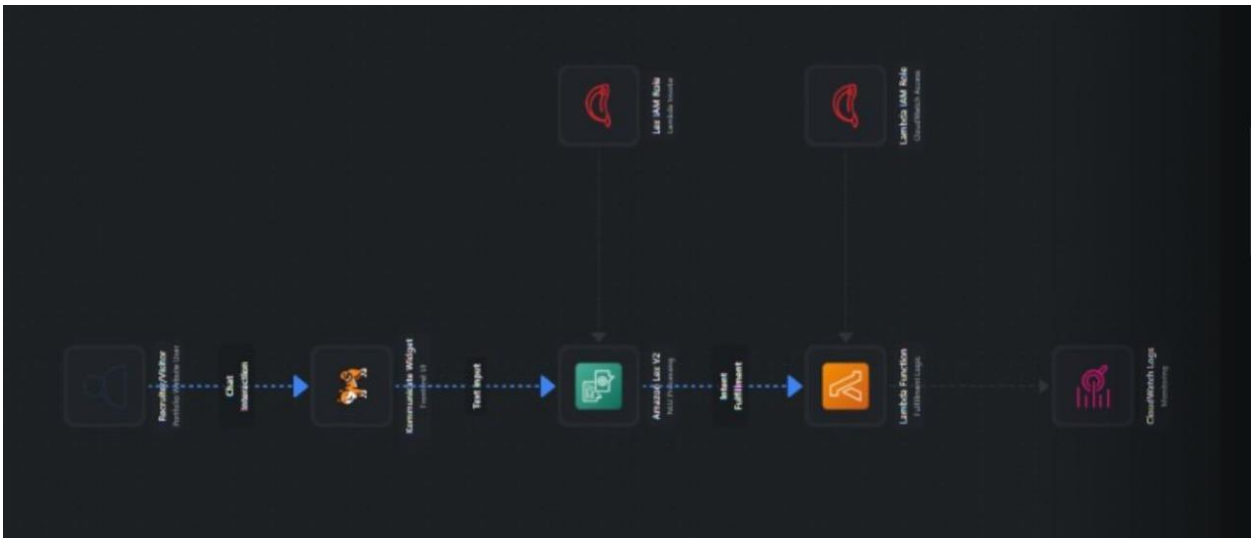
---

3. Project Objectives

- **Instant Availability:** Provide immediate answers to common recruitment questions (Skills, Certifications, Contact).
- **Serverless Architecture:** Utilize AWS Lambda to pay only for compute time used, ensuring zero cost when idle.
- **Natural Language Processing:** Use AWS Lex to understand varied human phrasing (e.g., "Show me your work" vs. "Projects").
- **Seamless Integration:** Embed the bot directly into the existing portfolio site using a custom JavaScript widget.

---

4. Architecture Overview



The infrastructure is divided into logical layers:

- **Frontend (The Interface):** The Portfolio Website hosts the **Kommunicate.io** chat widget, which acts as the user interface.
- **NLU Layer (The Brain): Amazon Lex V2** processes text input to determine the user's intent (e.g., "GetProjects").
- **Compute Layer (The Logic): AWS Lambda** executes Python code to fetch the requested data (links, text) and construct the response.
- **Integration Layer:** IAM Roles secure the communication between Lex and Lambda.

---

5. AWS Services Used

- **Amazon Lex V2:** For building the conversational interface and managing Intents.
- **AWS Lambda:** Serverless compute service running Python 3.9 to handle fulfillment.
- **AWS IAM:** Managed permissions to allow Lex to invoke Lambda securely.
- **CloudWatch:** Used for monitoring logs and debugging Lambda execution errors.

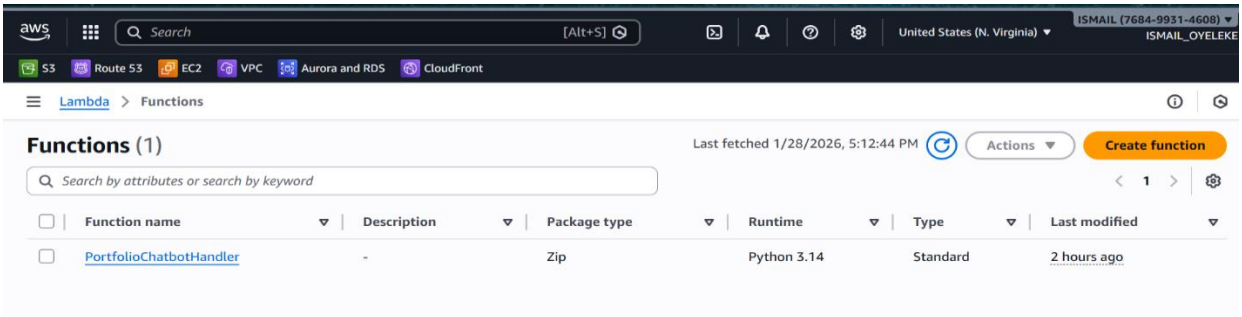
---

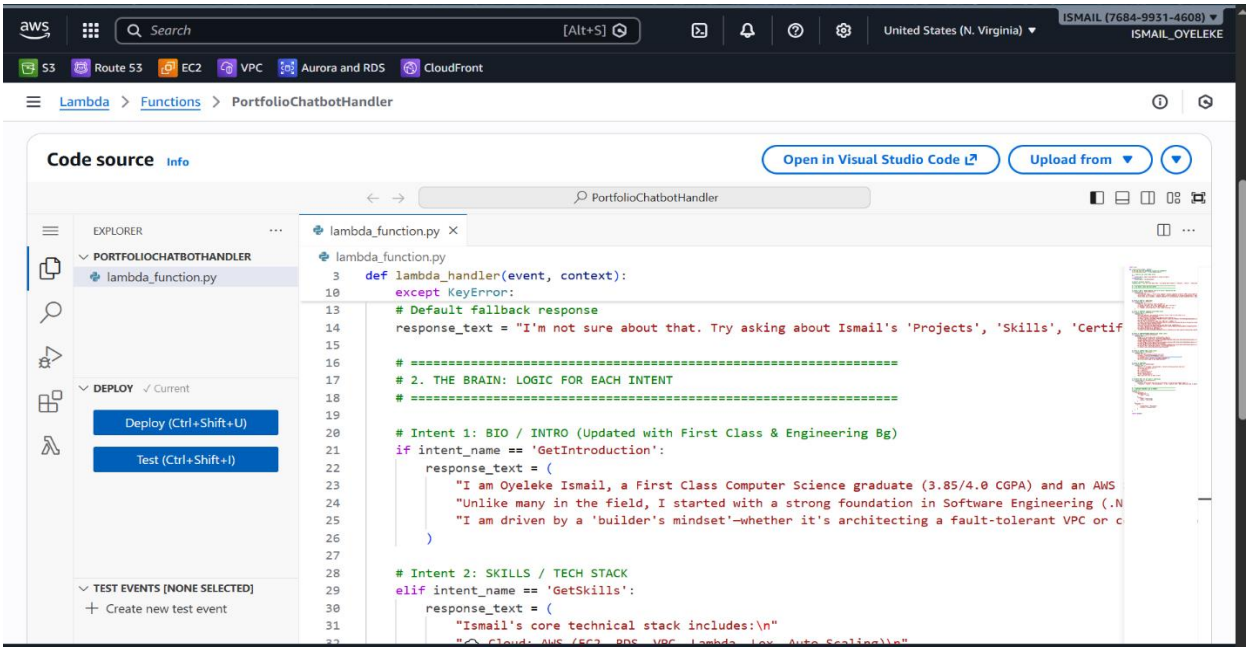
6. Implementation Steps

Step 1: Building the Brain (AWS Lambda)

I started by creating the backend logic using AWS Lambda. I selected **Python 3.14** as the runtime and authored a script, PortfolioChatbotHandler.

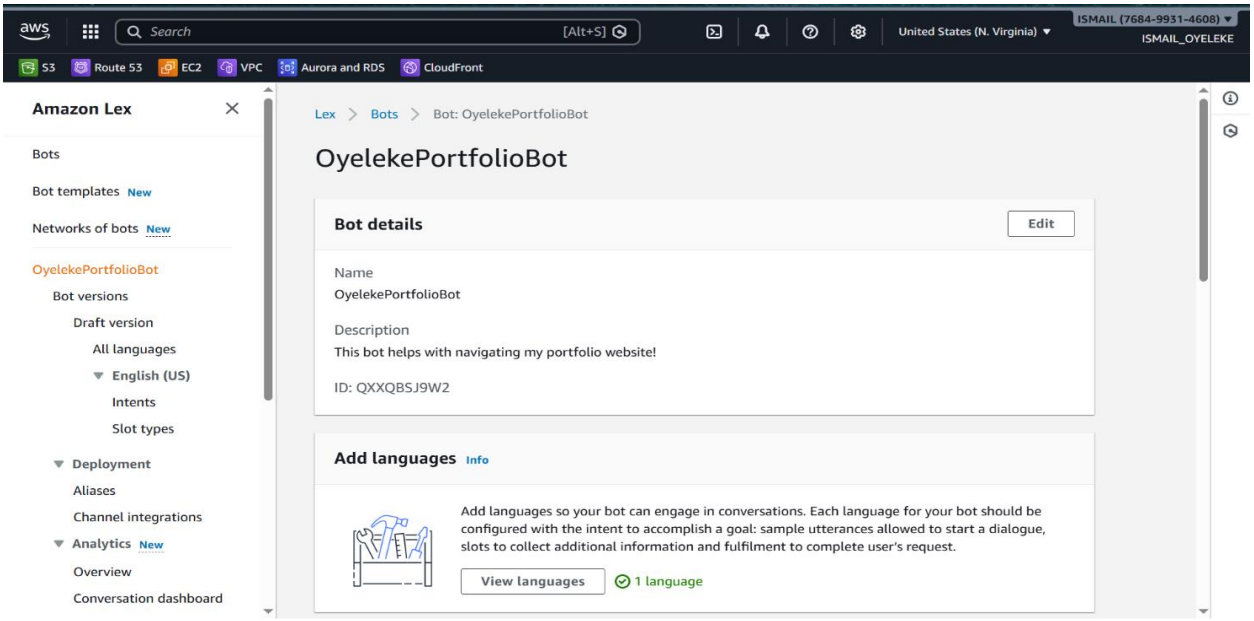
The script handles specific "Intents" (actions). For example, if the user triggers GetProjects, the Lambda function returns a formatted list of my top projects with GitHub links.





Step 2: Creating the Bot (Amazon Lex)

I provisioned a new bot named OyelekePortfolioBot in Amazon Lex V2. I configured it with **"English (US)"** and a high confidence threshold to ensure accuracy.

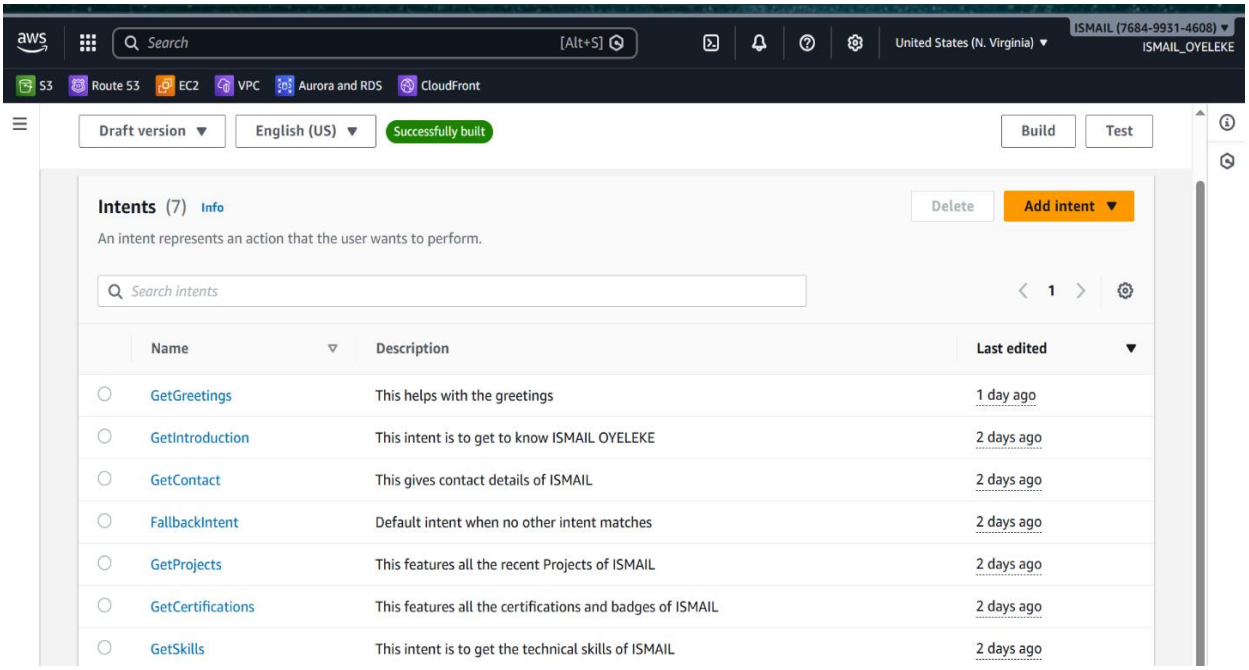


Step 3: Teaching the Bot (Intents & Utterances)

I defined 6 specific "Intents" to cover the most common recruiter questions. For each intent, I provided training data (Sample Utterances):

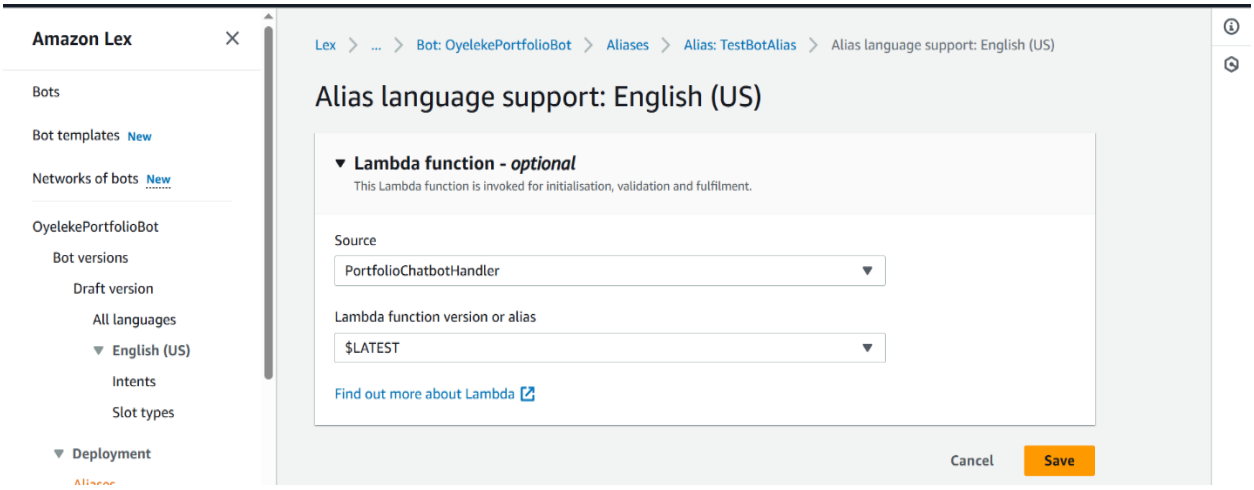
1. **GetIntroduction:** ("Who is Oyeleke?", "Bio")
2. **GetSkills:** ("What is your tech stack?", "Do you know AWS?")
3. **GetProjects:** ("Show me your portfolio", "GitHub")
4. **GetCertifications:** ("Are you certified?", "AWS Badges")
5. **GetContact:** ("How do I contact you?", "Email")
6. **GetGreetings:** ("Hello", "Hi", "Start")

For every intent, I enabled **"Use a Lambda function for fulfillment"** to connect the NLU layer to my Python backend.



Step 4: Connecting Lex to Lambda

I navigated to the Bot Aliases and linked the English (US) language version to my PortfolioChatbotHandler Lambda function. This established the bridge allowing Lex to send user input to the code for processing.



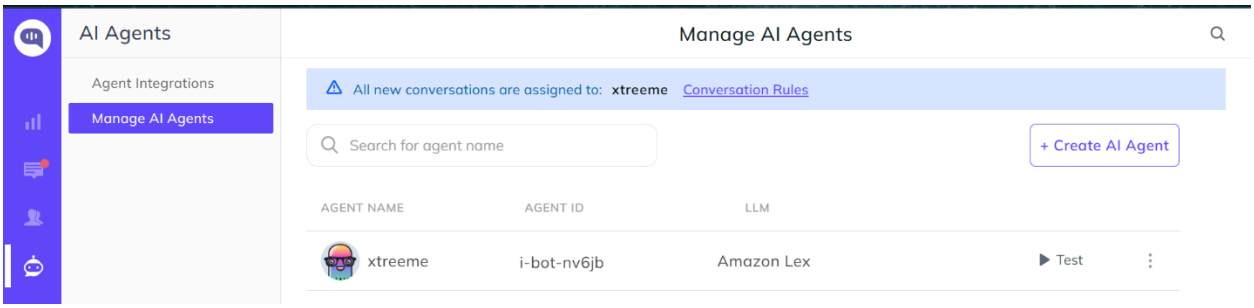
Step 5: Testing and Refinement

I used the built-in "Test" window in AWS Lex to verify the logic.

- **Test:** Typed "Who is Oyeleke?" → **Result:** Bot replied with the Bio from Lambda.
- **Test:** Typed "Pizza" → **Result:** Bot triggered the FallbackIntent ("I didn't quite catch that...").

Step 6: Website Integration (Kommunicate)

To display the bot on the web, I used **Kommunicate.io**. I created a "Service Account" user in AWS IAM with AmazonLexRunBotsOnly permissions to generate secure Access Keys. I then connected these keys to the Kommunicate dashboard.



Step 7: Deployment & The "Welcome" Trigger

I embedded the generated JavaScript code into the index.html of my portfolio. To ensure the bot engages users immediately, I added a custom onInit function with a setTimeout delay. This forces the bot to send a "Welcome" signal automatically when the widget loads, presenting the user with a menu of options instantly.

7. Security Considerations

- **IAM Roles:** I utilized the Principle of Least Privilege by creating a restricted IAM user for Kommunicate that only has permission to *Run* the bot (AmazonLexRunBotsOnly), protecting my main AWS account admin privileges.
- **No Hardcoded Secrets:** AWS credentials are stored securely in the integration platform, not in the frontend code.

---

8. Challenges and Solutions

Challenge: Overfitting (The "Favorite Food" Issue)

- **Issue:** The bot was confusing "What is your favorite food?" with "What is your stack?" because the sentence structures were identical.
- **Solution:** I increased the **NLU Confidence Threshold** from 0.40 to 0.70. This forced the bot to be strictly accurate and use the Fallback intent for irrelevant queries.

---

9. Project Outcome

I successfully deployed a fully functional AI assistant on ismailoyeleke.com. The bot correctly identifies user intent, serves dynamic content (links, lists) via Python, and handles errors gracefully. This project demonstrates proficiency in **Serverless Logic**, **Natural Language Understanding**, and **Frontend-Backend Integration**.

---

10. GitHub Repository

Here is the GitHub Repository for the Lambda logic and integration code: [https://github.com/ISMAIL-OYELEKE/Project-4-Serverless-Chatbot-Build-an-AI-powered-chatbot-with-AWS-Lex-Lambda]

---

11. Conclusion

This project moves beyond standard cloud infrastructure to explore **Applied AI**. By combining AWS Lex and Lambda, I created a tool that not only serves a practical purpose for recruitment but also showcases the power of event-driven, serverless architecture in creating interactive user experiences.